

ANSIBLE PARA DEVOPS

Guía completa de automatización, configuración y despliegue



Fundamentos | Playbooks | Roles | Vault | CI/CD
Linux | Windows | Cloud | Redes | Kubernetes

De lo básico a un proyecto profesional

Edición de estudio - Julio de 2026

PRESENTACIÓN

Acerca de esta guía

Material de estudio y referencia práctica para DevOps, infraestructura y operaciones.

Objetivo. Explicar Ansible desde sus fundamentos hasta un uso profesional: arquitectura, inventarios, playbooks, roles, seguridad, pruebas, integración CI/CD y casos reales en Linux, Windows, redes, nube y Kubernetes.

Enfoque. La guía combina conceptos, ejemplos ejecutables, buenas prácticas, advertencias y un proyecto de referencia. No reemplaza la documentación de la versión instalada; Ansible y sus colecciones evolucionan de manera independiente.

Idea central: Ansible convierte procedimientos manuales en automatizaciones repetibles, revisables y versionables. Su valor no está solo en ejecutar comandos, sino en describir el estado deseado de los sistemas.

Contenido

1. Parte I - Fundamentos: concepto, importancia, arquitectura e instalación
2. Parte II - Lenguaje de automatización: inventarios, módulos, playbooks y variables
3. Parte III - Reutilización y control: templates, handlers, roles, colecciones y Vault
4. Parte IV - Casos de uso: Linux, Windows, redes, cloud, Kubernetes y despliegues
5. Parte V - Calidad y operación profesional: testing, CI/CD, seguridad, rendimiento y troubleshooting
6. Parte VI - Proyecto completo, roadmap, libros, glosario y referencias

Referencias principales: [1], [2], [3]

PARTE I

Fundamentos de Ansible

Qué problema resuelve, por qué es importante y cómo trabaja.

1. ¿Qué es Ansible?

Ansible es un motor abierto de automatización de TI. Permite aprovisionar recursos, administrar configuraciones, desplegar aplicaciones, orquestar flujos y ejecutar tareas puntuales sobre uno o muchos sistemas.

- Es declarativo en gran parte de su uso: se indica el estado deseado y los módulos intentan alcanzarlo.
- Los playbooks se escriben en YAML y suelen leerse como procedimientos documentados.
- El nodo de control administra nodos remotos mediante SSH, PowerShell Remoting, WinRM, APIs y plugins.
- No requiere un agente permanente de Ansible ejecutándose en cada nodo administrado.
- Puede trabajar con servidores físicos, máquinas virtuales, dispositivos de red, nubes, contenedores y servicios de terceros.

Importante: Ansible no es solo una herramienta de “copiar archivos”. Es una plataforma de automatización y orquestación. Puede usarse como infraestructura como código, aunque su especialidad histórica es la configuración y la operación.

Referencias principales: [1], [2], [3]

2. Importancia dentro de DevOps

DevOps busca entregar cambios con rapidez sin perder confiabilidad. Para lograrlo, las tareas de infraestructura y operación deben ser reproducibles, medibles y revisables. Ansible aporta:

Necesidad DevOps	Aporte de Ansible
Automatización	Elimina pasos manuales repetitivos y reduce diferencias entre servidores.
Consistencia	Aplica la misma definición a desarrollo, pruebas, staging y producción.
Versionado	Playbooks, roles e inventarios pueden guardarse en Git y revisarse por pull request.
Velocidad	Permite operar muchos nodos en paralelo y reutilizar componentes.
Auditoría	Los cambios quedan asociados a código, commits, pipelines y resultados de ejecución.
Recuperación	Facilita reconstruir configuraciones y volver a aplicar el estado esperado.
Colaboración	Desarrollo, operaciones, seguridad y redes pueden compartir automatizaciones.

Ansible dentro de un flujo DevOps

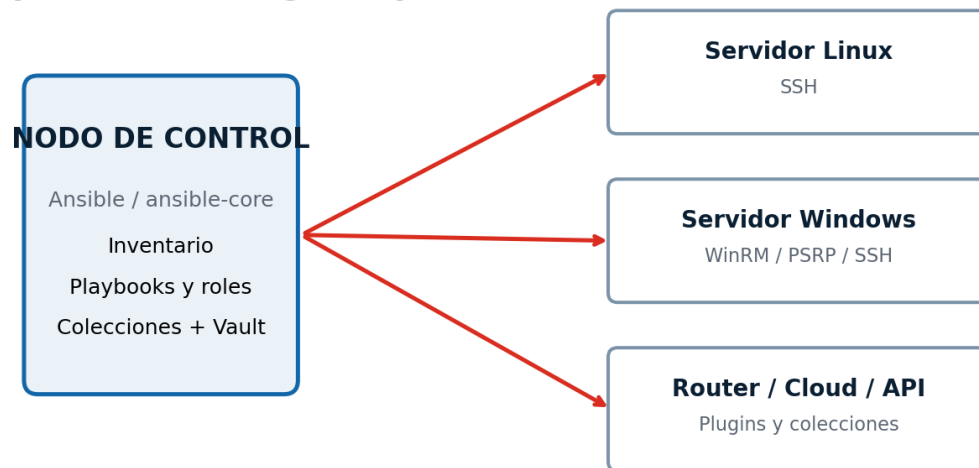


Automatización repetible, versionada y controlada por revisiones.

Referencias principales: [3], [4]

3. Arquitectura y funcionamiento

Arquitectura sin agente permanente en los nodos administrados



Ansible envía acciones, recibe resultados y deja el sistema en el estado deseado.

3.1 Componentes básicos

Componente	Función
Nodo de control	Equipo donde se instala y ejecuta Ansible.
Nodo administrado	Servidor, dispositivo o servicio que recibe la automatización.
Inventario	Fuente de hosts, grupos y variables.
Módulo	Unidad que realiza una acción concreta: instalar un paquete, copiar un archivo, crear un usuario.
Playbook	Plano de automatización en YAML compuesto por uno o más plays.
Plugin	Extensión para conexiones, filtros, inventarios, callbacks, caché y otras funciones.
Colección	Paquete distribuible que agrupa módulos, plugins, roles, playbooks y documentación.
Role	Estructura reutilizable de tareas, variables, handlers, templates y archivos.

3.2 Flujo de una ejecución

- Ansible carga su configuración y el inventario.
- Selecciona hosts según el patrón indicado.
- Resuelve variables, plugins, roles y colecciones.
- Establece la conexión al nodo o a la API.
- Ejecuta módulos o acciones y recibe resultados estructurados.
- Marca cada tarea como ok, changed, failed, skipped o unreachable.
- Ejecuta handlers notificados y presenta un resumen final.

Idempotencia: La mayoría de los módulos intentan evitar cambios cuando el sistema ya está en el estado deseado. Sin embargo, la idempotencia depende del módulo y de cómo se haya escrito el playbook; los comandos shell arbitrarios pueden romperla.

4. Instalación y primer laboratorio

4.1 ansible-core y paquete ansible

Paquete	Contenido	Cuándo elegirlo
ansible-core	Runtime, lenguaje, CLI y módulos/plugins integrados básicos.	Entornos controlados donde se instalarán solo las colecciones necesarias.
ansible	Paquete comunitario más amplio que agrega una selección de colecciones.	Aprendizaje y uso general con más contenido disponible desde el inicio.

Buena práctica: Use un entorno virtual de Python o pipx. Fije las versiones de Ansible y de las colecciones en proyectos que deban ser reproducibles.

BASH

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install ansible
ansible --version
```

En macOS y Linux el nodo de control es el uso más habitual. Para administrar Windows se puede utilizar un nodo de control Linux/macOS y conexiones WinRM, PSRP o SSH. El soporte y los requisitos deben verificarse contra la documentación de la versión instalada.

4.2 Laboratorio mínimo

INI

```
# inventory.ini
[lab]
servidor1 ansible_host=192.168.56.21

[lab:vars]
ansible_user=devops
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

BASH

```
ansible -i inventory.ini lab -m ansible.builtin.ping
ansible -i inventory.ini lab -m ansible.builtin.setup
```

No confundir: El módulo ping de Ansible no envía ICMP como el comando de red ping. Comprueba conectividad, autenticación y capacidad de ejecutar un módulo en el nodo.

Referencias principales: [5], [8], [9]

PARTE II

Inventarios, comandos y playbooks

El núcleo del trabajo diario con Ansible.

5. Inventarios

El inventario define qué sistemas administra Ansible y cómo se agrupan. Puede ser estático (INI o YAML) o dinámico mediante plugins que consultan una nube, CMDB u otra fuente.

YAML

```
all:
  children:
    web:
      hosts:
        web01:
          ansible_host: 10.10.1.21
        web02:
          ansible_host: 10.10.1.22
    database:
      hosts:
        db01:
          ansible_host: 10.10.2.31
  vars:
    ansible_user: devops
```

Elemento	Ejemplo	Descripción
Host	web01	Nombre lógico usado por Ansible.
ansible_host	10.10.1.21	Dirección real o DNS.
Grupo	web	Conjunto de hosts con función o entorno común.
Grupo padre	production	Agrupar otros grupos.
Variables	ansible_user	Datos de conexión o configuración.
Inventario dinámico	aws_ec2	Plugin que descubre recursos automáticamente.

BASH

```
ansible-inventory -i inventory.yml --graph
ansible-inventory -i inventory.yml --host web01
ansible -i inventory.yml 'web:&production' -m ansible.builtin.ping
```

Organización: En proyectos reales conviene separar inventarios por entorno y mantener variables en `group_vars` y `host_vars`. Los secretos no deben quedar en texto plano.

Referencias principales: [10], [11]

6. Comandos ad hoc

Los comandos ad hoc ejecutan una acción rápida sin crear un playbook. Son útiles para diagnóstico, consultas o tareas excepcionales; no sustituyen automatizaciones versionadas.

Objetivo	Comando
Probar conectividad	<code>ansible all -m ansible.builtin.ping</code>
Consultar uso de disco	<code>ansible all -m ansible.builtin.command -a 'df -h'</code>
Instalar un paquete	<code>ansible web -b -m ansible.builtin.package -a 'name=nginx'</code>

Objetivo	Comando
	state=present'
Reiniciar un servicio	ansible web -b -m ansible.builtin.service -a 'name=nginx state=restarted'
Copiar un archivo	ansible web -b -m ansible.builtin.copy -a 'src=motd dest=/etc/motd'
Reunir facts	ansible all -m ansible.builtin.setup

Regla práctica: Si una tarea debe repetirse, auditarse o ejecutarse en producción, conviértala en playbook o role.

7. Playbooks

Un playbook es un archivo YAML que relaciona un conjunto de hosts con tareas, variables, roles y políticas de ejecución.

YAML

```
---
- name: Configurar servidores web
  hosts: web
  become: true
  vars:
    paquete_web: nginx

  tasks:
    - name: Instalar servidor web
      ansible.builtin.package:
        name: "{{ paquete_web }}"
        state: present

    - name: Publicar página inicial
      ansible.builtin.copy:
        content: "Servidor administrado por Ansible"
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: "0644"

    - name: Asegurar que Nginx esté iniciado
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

BASH

```
ansible-playbook -i inventory.yml site.yml --syntax-check
ansible-playbook -i inventory.yml site.yml --check --diff
ansible-playbook -i inventory.yml site.yml
```

Resultado	Significado
ok	La tarea fue correcta y no necesitó cambios.
changed	La tarea modificó el estado del nodo.
failed	La tarea falló en ese host.
unreachable	Ansible no pudo conectarse al host.
skipped	La tarea no se ejecutó por una condición o tag.
rescued	Un bloque de recuperación trató el error.
ignored	El error fue ignorado explícitamente.

Referencias principales: [12], [13]

8. Módulos y FQCN

Los módulos representan operaciones de alto nivel. Siempre que exista un módulo adecuado, suele ser preferible a command o shell porque puede validar estado, soportar check mode y ofrecer resultados estructurados.

Área	Módulos frecuentes
Paquetes	ansible.builtin.package, apt, dnf, pip
Archivos	copy, template, file, lineinfile, blockinfile, synchronize

Área	Módulos frecuentes
Servicios	service, systemd_service
Usuarios	user, group, authorized_key
Comandos	command, shell, script, raw
Datos	set_fact, debug, assert, uri
Windows	ansible.windows.win_package, win_service, win_copy, win_updates
Kubernetes	kubernetes.core.k8s, k8s_info
Cloud	Colecciones específicas del proveedor

FQCN: La documentación recomienda nombres totalmente calificados, por ejemplo `ansible.builtin.copy`, para identificar la colección, enlazar mejor la documentación y evitar colisiones de nombres.

Referencias principales: [14], [15]

9. Variables, facts y precedencia

Las variables permiten reutilizar automatizaciones. Pueden definirse en inventario, group_vars, host_vars, playbooks, roles, archivos externos o línea de comandos. Cuando la misma variable aparece en varios lugares, se aplican reglas de precedencia.

YAML

```
# group_vars/web.yml
app_name: portal
app_port: 8080
paquetes_base:
  - curl
  - git
  - unzip
```

YAML

```
- name: Mostrar información del host
  ansible.builtin.debug:
    msg: >-
      {{ inventory_hostname }} ejecuta
      {{ ansible_facts.distribution }}
      {{ ansible_facts.distribution_version }}
```

- Use nombres claros y consistentes.
- Coloque valores por defecto en defaults/main.yml de una role.
- Evite depender innecesariamente de variables extra (-e), porque tienen alta precedencia.
- No mezcle secretos con variables normales sin cifrado o un gestor externo.
- Documente variables obligatorias y válidelas con ansible.builtin.assert.

Referencias principales: [16]

10. Condiciones, bucles, register y filtros

YAML

```
- name: Instalar paquetes base
  ansible.builtin.package:
    name: "{{ item }}"
    state: present
    loop: "{{ paquetes_base }}"

- name: Consultar estado HTTP
  ansible.builtin.uri:
    url: "http://localhost:{{ app_port }}/health"
    return_content: true
    register: health
    changed_when: false

- name: Validar respuesta
  ansible.builtin.assert:
    that:
      - health.status == 200
    fail_msg: "El servicio no está saludable"
    when: ansible_facts.os_family == "Debian"
```

Recurso	Uso
when	Ejecuta una tarea solo si se cumple una expresión.
loop	Repite una tarea para cada elemento.
register	Guarda el resultado de una tarea en una variable.
changed_when	Controla cuándo una tarea se informa como changed.
failed_when	Define condiciones personalizadas de fallo.
default	Proporciona un valor alternativo si una variable no existe.
map / select / reject	Transforma y filtra listas.
json_query	Consulta estructuras complejas cuando la colección/filtro está disponible.

PARTE III

Reutilización, configuración y secretos

Cómo pasar de playbooks pequeños a automatizaciones mantenibles.

11. Templates con Jinja2

El módulo `template` genera archivos a partir de plantillas Jinja2. Permite insertar variables, condiciones y bucles manteniendo una fuente común para distintos entornos.

JINJA2

```
# templates/nginx.conf.j2
server {
    listen {{ http_port }};
    server_name {{ server_name }};

    location / {
        proxy_pass http://127.0.0.1:{{ app_port }};
        proxy_set_header Host $host;
    }
}
```

YAML

```
- name: Publicar configuración de Nginx
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/conf.d/app.conf
    owner: root
    group: root
    mode: "0644"
    validate: "nginx -t -c %s"
    notify: Reiniciar Nginx
```

Validación: Cuando el módulo lo permita, valide el archivo antes de reemplazar la configuración activa. Esto evita desplegar archivos sintácticamente inválidos.

12. Handlers

Los handlers son tareas que se ejecutan cuando otra tarea informa un cambio y los notifica. Se usan para reiniciar o recargar servicios solo cuando corresponde.

YAML

```
handlers:
- name: Reiniciar Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

Ventaja: Si el archivo no cambia, el handler no se ejecuta. Esto reduce interrupciones y hace la automatización más idempotente.

Referencias principales: [17]

13. Roles

Una role organiza contenido relacionado en una estructura conocida y reutilizable.

TEXT

```
roles/
├── nginx/
│   ├── defaults/main.yml
│   └── vars/main.yml
```

```
├── tasks/main.yml
├── handlers/main.yml
├── templates/
├── files/
├── meta/main.yml
└── README.md
```

Directorio	Propósito
defaults	Valores predeterminados con baja precedencia.
vars	Variables internas de la role con mayor precedencia.
tasks	Tareas principales.
handlers	Acciones notificadas.
templates	Plantillas Jinja2.
files	Archivos estáticos.
meta	Metadatos, dependencias y compatibilidad.
README.md	Documentación, variables, ejemplos y requisitos.

YAML

```
- name: Configurar plataforma web
  hosts: web
  become: true
  roles:
    - role: nginx
      vars:
        http_port: 80
```

Referencias principales: [18], [19]

14. Colecciones y Ansible Galaxy

Las colecciones son la unidad moderna de distribución de contenido. Pueden contener módulos, plugins, roles, playbooks y documentación. Galaxy permite descubrir e instalar contenido comunitario; Automation Hub ofrece contenido para entornos empresariales.

YAML

```
# requirements.yml
collections:
  - name: community.general
    version: "13.2.0"
  - name: kubernetes.core
    version: "6.5.0"

roles:
  - name: geerlingguy.nginx
    version: "3.2.0"
```

BASH

```
ansible-galaxy collection install -r requirements.yml
ansible-galaxy collection list
ansible-galaxy role install -r requirements.yml
```

Reproducibilidad: Fije versiones y revise la procedencia del contenido. Una actualización automática de una colección puede cambiar comportamiento, dependencias o compatibilidad.

Referencias principales: [20], [21]

15. Ansible Vault y gestión de secretos

Ansible Vault cifra archivos o variables para evitar secretos en texto plano dentro del repositorio.

BASH

```
ansible-vault create group_vars/production/vault.yml
ansible-vault edit group_vars/production/vault.yml
ansible-vault encrypt_string 'valor-secreto' --name 'db_password'
ansible-playbook site.yml --ask-vault-pass
```

- No guarde contraseñas reales en inventarios sin cifrar.
- No incluya la contraseña de Vault en el repositorio.
- Use identidades de Vault cuando haya varios entornos o equipos.
- Considere gestores externos de secretos para rotación, auditoría y acceso dinámico.
- Use `no_log: true` para evitar exponer datos sensibles en la salida, pero recuerde que reduce la capacidad de diagnóstico.

Alcance: Vault protege datos almacenados. No reemplaza controles de acceso, rotación de credenciales, cifrado de transporte ni un gestor empresarial de secretos.

16. Control de ejecución y manejo de errores

Recurso	Finalidad
tags	Ejecutar o saltar partes de un playbook.
--limit	Restringir la ejecución a hosts o grupos.
--check	Simular cambios en módulos compatibles.
--diff	Mostrar diferencias de archivos o plantillas.
serial	Actualizar hosts por lotes.
max_fail_percentage	Detener una actualización cuando fallan demasiados hosts.
any_errors_fatal	Propagar un fallo no tratado y finalizar el play.
block / rescue / always	Agrupar tareas y definir recuperación similar a excepciones.
ignore_errors	Continuar tras ciertos fallos; debe usarse con criterio.
delegate_to	Ejecutar una tarea en otro nodo, por ejemplo un balanceador.

YAML

```
- name: Despliegue controlado
  block:
    - name: Publicar nueva versión
      ansible.builtin.copy:
        src: app.tar.gz
        dest: /opt/app/app.tar.gz

    - name: Validar servicio
      ansible.builtin.uri:
        url: http://localhost:8080/health
        status_code: 200

  rescue:
    - name: Restaurar versión anterior
      ansible.builtin.command: /opt/app/rollback.sh
      changed_when: true

  always:
    - name: Registrar finalización
      ansible.builtin.debug:
        msg: "Proceso de despliegue finalizado"
```

Check mode: Es una ayuda, no una prueba absoluta. Algunos módulos no lo soportan completamente y una simulación no reproduce todos los efectos de una ejecución real.

Referencias principales: [22], [23], [24]

PARTE IV

Casos de uso en infraestructura y operaciones

Aplicaciones prácticas en distintos dominios.

17. Administración de Linux

- Usuarios, grupos, claves SSH y sudo.
- Paquetes, repositorios y actualizaciones.
- Servicios systemd y procesos.
- Archivos, permisos, propietarios y plantillas.
- Firewall, Nginx/Apache, cron, logs y monitoreo.
- Montajes, almacenamiento, backups y agentes de seguridad.

YAML

```
- name: Crear usuario de despliegue
  ansible.builtin.user:
    name: deploy
    groups: sudo
    append: true
    shell: /bin/bash

- name: Instalar clave autorizada
  ansible.posix.authorized_key:
    user: deploy
    key: "{{ lookup('file', 'files/deploy.pub') }}"

- name: Endurecer permisos de SSH
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PasswordAuthentication'
    line: 'PasswordAuthentication no'
    validate: '/usr/sbin/sshd -t -f %s'
  notify: Reiniciar SSH
```

Precaución: Pruebe cambios de acceso remoto en un entorno seguro y mantenga una sesión alternativa antes de cerrar la autenticación por contraseña.

18. Administración de Windows

Ansible puede administrar Windows mediante colecciones específicas y conexiones WinRM, PSRP o SSH. Los módulos de Windows suelen comenzar con win_.

YAML

```
- name: Configurar servidores Windows
  hosts: windows
  tasks:
    - name: Instalar IIS
      ansible.windows.win_feature:
        name: Web-Server
        state: present
        include_management_tools: true

    - name: Copiar página
      ansible.windows.win_copy:
        content: "Servidor administrado por Ansible"
        dest: C:\inetpub\wwwroot\index.html

    - name: Asegurar servicio IIS
      ansible.windows.win_service:
        name: W3SVC
        state: started
        start_mode: auto
```

- Active Directory y usuarios mediante colecciones apropiadas.
- Features de Windows Server, servicios, registro y firewall.
- Paquetes MSI o gestores como Chocolatey.
- Windows Update y reinicios controlados.
- Autenticación Kerberos, certificados o claves según el transporte.

Referencias principales: [25], [26], [27]

19. Automatización de redes

Ansible puede gestionar dispositivos de red mediante colecciones de fabricantes y plugins de conexión. Es útil para cambios masivos, backups de configuración, cumplimiento y despliegue de políticas.

Caso	Ejemplo
Configuración	Interfaces, VLAN, routing, ACL y NTP.
Backups	Capturar configuraciones antes de un cambio.
Validación	Comprobar estado operativo y comparar con políticas.
Cambios controlados	Aplicar por lotes, usar check mode cuando sea compatible y validar.
Inventario dinámico	Sincronizar dispositivos desde NetBox u otras fuentes.
Auditoría	Guardar resultados y diferencias en sistemas centrales.

Diferencia: En dispositivos de red la ejecución puede ocurrir localmente en el nodo de control y comunicarse con el equipo mediante CLI o API, en lugar de copiar módulos al dispositivo.

20. Cloud y Kubernetes

Las colecciones extienden Ansible hacia proveedores cloud, plataformas de virtualización y Kubernetes. Las credenciales, dependencias y versiones varían por colección.

YAML

```
- name: Crear namespace de Kubernetes
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: v1
      kind: Namespace
      metadata:
        name: plataforma
```

- Crear o modificar recursos mediante APIs.
- Configurar sistemas después del aprovisionamiento.
- Desplegar manifiestos y operadores en Kubernetes.
- Coordinar acciones entre nube, balanceadores, DNS y aplicaciones.
- Consultar inventarios dinámicos en AWS, Azure, GCP, VMware u OpenStack.

Referencias principales: [20], [28]

21. Despliegue de aplicaciones

Ansible puede coordinar despliegues simples o actualizaciones progresivas. Un patrón de rolling update reduce el impacto al retirar temporalmente cada nodo del balanceador, actualizarlo, verificarlo y reincorporarlo.

14. Seleccionar un lote pequeño con serial.
15. Retirar el nodo del balanceador mediante `delegate_to`.
16. Copiar artefactos o desplegar contenedores.
17. Ejecutar migraciones compatibles.
18. Reiniciar o recargar servicios.
19. Comprobar health checks.
20. Reincorporar el nodo y continuar con el siguiente lote.

Diseño seguro: La automatización debe incluir validación, rollback, tiempos de espera y criterios de detención. Automatizar un procedimiento inseguro solo hace que falle más rápido.

22. Integración con CI/CD

Los pipelines pueden validar y ejecutar contenido de Ansible. En producción conviene separar la fase de verificación de la fase de aplicación y requerir aprobaciones cuando el riesgo lo justifique.

YAML

```
name: Validar Ansible
on: [push, pull_request]

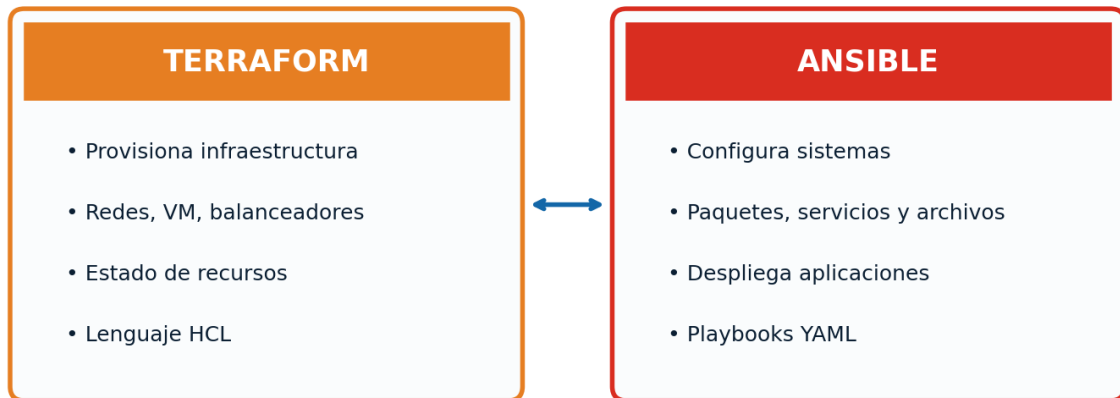
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install ansible-core ansible-lint yamllint
      - run: yamllint .
      - run: ansible-lint
      - run: ansible-playbook -i inventories/test/hosts.yml site.yml --syntax-check
```

Etapas	Control
Pull request	Revisión de código, lint, sintaxis y pruebas.
Build	Generación de artefactos y fijación de versiones.
Staging	Ejecución completa en entorno representativo.
Aprobación	Control humano o política para producción.
Producción	Límites, lotes, credenciales protegidas y logs.
Postdespliegue	Smoke tests, métricas, alertas y rollback.

Referencias principales: [29], [30], [31]

23. Ansible y Terraform

Terraform y Ansible: herramientas complementarias



Patrón frecuente: Terraform crea la infraestructura y Ansible la prepara para ejecutar aplicaciones.

Pregunta	Terraform	Ansible
¿Qué describe?	Infraestructura y recursos.	Configuración y procedimientos operativos.
Lenguaje habitual	HCL.	YAML + Jinja2.
Estado	Mantiene state de recursos.	No usa un state global equivalente para la mayoría de tareas.
Fortaleza	Provisionamiento y ciclo de vida de recursos.	Configuración, despliegue y orquestación.
Flujo combinado	Crea redes, VM, balanceadores y bases.	Instala paquetes, configura servicios y despliega aplicaciones.

Calidad, seguridad y operación profesional

Cómo mantener automatizaciones confiables y escalables.

24. Pruebas y calidad

Herramienta / técnica	Función
yamllint	Detecta problemas de sintaxis, formato e indentación YAML.
Ansible Lint	Aplica reglas de calidad y prácticas recomendadas a playbooks y roles.
--syntax-check	Valida la sintaxis del playbook.
--check --diff	Simula cambios y muestra diferencias cuando hay soporte.
Molecule	Prueba roles, playbooks y colecciones en escenarios reproducibles.
assert	Valida precondiciones y resultados durante la ejecución.
CI/CD	Ejecuta controles automáticamente ante cada cambio.
Entorno staging	Prueba integración real antes de producción.

BASH

```
python -m pip install ansible-dev-tools
ansible-lint
ansible-playbook site.yml --syntax-check
molecule test
```

Pirámide práctica: Combine validaciones rápidas en cada commit, pruebas Molecule para roles y pruebas de integración en staging. Ninguna herramienta aislada garantiza que un cambio sea seguro en producción.

Referencias principales: [29], [30], [31], [32]

25. Seguridad

- Aplicar mínimo privilegio y usar become solo donde sea necesario.
- Autenticar con claves, certificados o mecanismos corporativos; proteger claves privadas.
- Mantener host key checking y verificar la identidad de los nodos.
- Cifrar secretos y restringir quién puede descifrarlos.
- Evitar shell/command cuando exista un módulo específico.
- Fijar y revisar versiones de colecciones y dependencias.
- Usar FQCN, permisos explícitos y modos de archivo entre comillas.
- Validar plantillas y archivos antes de reemplazar configuraciones.
- Evitar no_log indiscriminado; oculta información útil para auditoría y diagnóstico.
- Revisar automatizaciones como código y ejecutar análisis estático.
- No incluir datos sensibles en logs, artefactos de CI o mensajes debug.
- Mantener backups y procedimientos de reversión.

Riesgo IaC: Una credencial o una configuración insegura puede propagarse a cientos de sistemas. La revisión y el testing son controles de seguridad, no solo de calidad.

26. Rendimiento y escalabilidad

Técnica	Uso y precaución
forks	Aumenta concurrencia; debe adaptarse a CPU, red y límites de los sistemas.
serial	Procesa hosts por lotes para reducir riesgo.
strategy	Controla cómo avanzan hosts y tareas.
Fact gathering	Desactívelo cuando no se utilicen facts; considere caché.
async / poll	Ejecuta tareas largas sin bloquear de la misma forma.
pipelining	Puede reducir conexiones y transferencias; requiere validar compatibilidad y seguridad.
Inventario dinámico	Evita listas manuales, pero puede requerir caché para no consultar APIs repetidamente.
Roles pequeños	Facilitan pruebas y reutilización; evite roles monolíticas.
Módulos idempotentes	Reducen trabajo innecesario frente a scripts arbitrarios.
Execution Environments	Empaquetan runtime, colecciones y dependencias para reproducibilidad.

Medición: No optimice a ciegas. Mida duración por tarea, latencia de conexión, número de hosts y carga en los sistemas administrados.

27. Troubleshooting

Método: Aísle el problema: conexión, inventario, variables, módulo, permisos, estado remoto y lógica. Cambie una cosa por vez y conserve la evidencia de la ejecución.

Síntoma	Causas frecuentes	Diagnóstico
UNREACHABLE	DNS, firewall, SSH/WinRM, usuario o clave.	-vvvv, ssh manual, Test-WSMan, puertos y credenciales.
Variable indefinida	Nombre incorrecto, precedencia o archivo no cargado.	debug, ansible-inventory --host, revisar group_vars/host_vars.
Módulo no encontrado	Colección faltante o FQCN incorrecto.	ansible-galaxy collection list, requirements.yml.
Permiso denegado	Falta become, usuario incorrecto o permisos de archivo.	whoami, become, sudoers, owner/group/mode.
Siempre changed	Uso de shell o changed_when incorrecto.	Reemplazar por módulo idempotente, revisar salida registrada.
Handler no corre	La tarea no cambió o no notificó el nombre correcto.	Revisar changed, notify y orden de handlers.
Check mode engañoso	Módulo sin soporte completo o dependencias entre tareas.	Probar staging y leer atributos del módulo.
Ejecución lenta	Facts, conexiones, API, forks o tareas repetidas.	Profile tasks, caché, forks y reducción de tareas.

BASH

```
ansible-playbook -i inventory.yml site.yml -vvvv
ansible-inventory -i inventory.yml --graph
ansible-config dump --only-changed
ansible-doc ansible.builtin.copy
ansible-galaxy collection list
```


PARTE VI

Proyecto profesional y plan de estudio

Una estructura completa para practicar y crecer.

28. Estructura recomendada de proyecto

TEXT

```

ansible-project/
├── ansible.cfg
├── requirements.yml
├── site.yml
├── inventories/
│   ├── development/
│   │   ├── hosts.yml
│   │   ├── group_vars/
│   │   └── host_vars/
│   └── production/
│       ├── hosts.yml
│       ├── group_vars/
│       └── host_vars/
├── playbooks/
│   ├── deploy.yml
│   ├── backup.yml
│   └── patch.yml
├── roles/
│   ├── common/
│   ├── nginx/
│   └── application/
├── molecule/
├── files/
├── templates/
├── .ansible-lint
├── .yamllint
└── README.md

```

- Separe datos de entorno de la lógica reutilizable.
- Documente requisitos, variables y ejemplos de ejecución.
- Incluya requirements.yml y fije versiones.
- No suba claves, contraseñas, inventarios sensibles ni archivos de Vault password.
- Use convenciones de nombres y FQCN.
- Agregue pruebas y pipeline desde el inicio.

29. Proyecto guiado: plataforma web

Objetivo: configurar dos servidores Linux con Nginx, desplegar una aplicación, aplicar variables por entorno, proteger secretos y validar el servicio.

21. Crear dos máquinas virtuales con acceso SSH por clave.
22. Definir inventarios development y production.
23. Crear role common para paquetes y usuarios base.
24. Crear role nginx con template y handler.
25. Crear role application para artefacto, servicio y health check.
26. Cifrar credenciales con Vault o integrar un gestor externo.
27. Ejecutar ansible-lint, syntax-check y Molecule.
28. Probar en development, luego staging y finalmente production con serial.
29. Integrar el flujo con GitHub Actions, GitLab CI o Jenkins.
30. Agregar métricas y smoke tests después del despliegue.

Resultado esperado: El repositorio debe permitir reconstruir y verificar la configuración sin depender de instrucciones manuales ocultas.

30. Roadmap de aprendizaje

Ruta de aprendizaje de Ansible



Nivel	Competencias	Proyecto sugerido
Inicial	Linux, SSH, YAML, inventario y comandos ad hoc.	Configurar una VM y publicar una página.
Básico	Playbooks, módulos, variables, facts, handlers.	Servidor Nginx reproducible.
Intermedio	Roles, templates, Vault, tags, blocks y collections.	Aplicación de tres capas con roles.
Profesional	Lint, Molecule, CI/CD, inventario dinámico y rolling updates.	Pipeline con staging y producción.
Especialización	Windows, redes, cloud, Kubernetes o Automation Platform.	Automatización de un entorno real del área elegida.

31. Libros recomendados

Libro	Nivel	Fortaleza principal
Ansible for DevOps - Jeff Geerling	Inicial a intermedio	Enfoque práctico para administrar servidores y aplicar Ansible en DevOps.
Ansible: Up and Running, 3rd Edition - Meijer, Hochstein y Moser	Inicial a avanzado	Referencia amplia sobre playbooks, inventarios, roles, cloud, redes y prácticas modernas.
The Ansible Workshop - Aymen El Amri	Inicial a avanzado	Laboratorios progresivos y ejercicios.
Mastering Ansible, 4th Edition - Freeman y Keating	Intermedio a avanzado	Arquitectura, extensiones, depuración y automatización compleja.
Ansible for Real-Life Automation - Gineesh Madapparambath	Intermedio	Casos reales en sistemas, cloud, redes y contenedores.
Ansible: Administre la configuración de sus servidores... - Yannig Perré	Inicial a intermedio	Alternativa en español orientada a administración de sistemas.

Cómo estudiar: Use un libro como ruta, pero verifique módulos, parámetros y compatibilidad en la documentación correspondiente a la versión instalada.

Referencias principales: [33], [34], [35], [36], [37]

32. Glosario esencial

Término	Definición
Ad hoc	Comando puntual ejecutado sin playbook.
Agentless	Arquitectura sin un agente permanente de Ansible en el nodo administrado.
Become	Mecanismo de escalada de privilegios.
Check mode	Simulación de cambios en módulos compatibles.
Collection	Paquete de módulos, plugins, roles y contenido.
Control node	Equipo desde el cual se ejecuta Ansible.
Diff mode	Muestra diferencias previstas o realizadas.
Fact	Información descubierta sobre un nodo.
FQCN	Nombre completo namespace.collection.recurso.
Handler	Tarea ejecutada cuando es notificada por un cambio.
Idempotencia	Propiedad de alcanzar el mismo estado sin cambios innecesarios al repetir.
Inventory	Fuente de hosts, grupos y variables.
Jinja2	Motor de plantillas y expresiones.
Managed node	Sistema administrado por Ansible.
Module	Unidad que realiza una acción.
Molecule	Framework de pruebas para contenido Ansible.
Play	Asociación entre hosts y tareas.
Playbook	Plano de automatización en YAML.
Plugin	Extensión de conexión, filtro, inventario, callback u otra función.
Role	Estructura reutilizable de tareas y recursos.
Tag	Etiqueta para seleccionar tareas.
Vault	Cifrado de secretos o archivos.
Variable precedence	Reglas que deciden qué valor gana cuando hay definiciones múltiples.

33. Referencias y fuentes

Documentación consultada en julio de 2026. Verifique siempre la documentación que corresponda a la versión instalada.

- [1] Ansible Community Documentation - Introduction to Ansible. [Consultar fuente](#)
- [2] Ansible Collaborative - What is Ansible?. [Consultar fuente](#)
- [3] Ansible project repository. [Consultar fuente](#)
- [4] Red Hat Ansible Automation Platform. [Consultar fuente](#)
- [5] Installing Ansible. [Consultar fuente](#)
- [6] Ansible concepts. [Consultar fuente](#)
- [7] Connection plugins. [Consultar fuente](#)
- [8] Installation Guide. [Consultar fuente](#)
- [9] Installing Ansible on specific operating systems. [Consultar fuente](#)
- [10] How to build your inventory. [Consultar fuente](#)
- [11] Working with dynamic inventory. [Consultar fuente](#)
- [12] Using Ansible playbooks. [Consultar fuente](#)
- [13] Working with playbooks. [Consultar fuente](#)
- [14] Collection Index. [Consultar fuente](#)
- [15] ansible.builtin.package module. [Consultar fuente](#)
- [16] Using variables. [Consultar fuente](#)
- [17] Handlers. [Consultar fuente](#)
- [18] Roles. [Consultar fuente](#)
- [19] Reusing Ansible artifacts. [Consultar fuente](#)
- [20] Installing collections. [Consultar fuente](#)
- [21] Ansible automation hub. [Consultar fuente](#)
- [22] Check mode and diff mode. [Consultar fuente](#)
- [23] Blocks. [Consultar fuente](#)
- [24] Error handling in playbooks. [Consultar fuente](#)
- [25] Managing Windows hosts. [Consultar fuente](#)
- [26] Windows Remote Management. [Consultar fuente](#)
- [27] Windows SSH. [Consultar fuente](#)
- [28] kubernetes.core.k8s module. [Consultar fuente](#)
- [29] Other tools and programs. [Consultar fuente](#)
- [30] Ansible Lint usage. [Consultar fuente](#)
- [31] Ansible Molecule. [Consultar fuente](#)
- [32] Testing - Ansible Development Tools. [Consultar fuente](#)
- [33] Ansible for DevOps - Leanpub. [Consultar fuente](#)
- [34] Ansible: Up and Running, 3rd Edition - O'Reilly. [Consultar fuente](#)
- [35] Mastering Ansible, 4th Edition - Packt. [Consultar fuente](#)
- [36] Ansible for Real-Life Automation - Packt. [Consultar fuente](#)
- [37] Ansible - Ediciones ENI. [Consultar fuente](#)

Cierre: La competencia profesional no consiste en memorizar módulos, sino en diseñar automatizaciones seguras, legibles, probadas y recuperables.